

Título: Projeto 1 — Benchmark de Suavização/Desfoque Gaussiano Iterativo: Implementação sequencial e paralela em memória compartilhada com OpenMP.

Autores: Lucas Luige Costa Miranda, Henrique Cerqueira Fadigas

Disciplina: DEC107 — Processamento Paralelo

Data: Maio de 2026

Sumário

1	Introdução	2
2	Metodologia	2
2.1	Implementação sequencial	2
2.2	Implementação paralela com OpenMP	3
2.2.1	OpenMP simples	3
2.2.2	Tiling (blocos 2D)	3
2.3	Hardware utilizado	3
2.4	Medição de tempo e métricas	4
3	Resultados	4
3.1	Teste de corretude	4
3.2	Tempos seriais	4
3.3	OpenMP — Tempos, Speedup e Eficiência	5
3.4	Tiling — Tempos, Speedup e Eficiência	5
3.5	Gráficos	6
3.6	Balanceamento de carga e escalabilidade	9
4	Discussão	9
4.1	Corretude das implementações	9
4.2	Comparação serial vs paralela	9
4.3	Impacto do número de <i>threads</i>	10
4.4	Gargalos e limitações	10
4.5	Sugestões de melhorias	10
5	Conclusão	10
6	Referências	11

1 Introdução

O objetivo deste projeto é implementar um benchmark de suavização Gaussiana iterativa sobre imagens 2D e comparar o desempenho de uma versão serial com versões paralelas usando OpenMP. A ideia é entender na prática o quanto o paralelismo ajuda (e onde ele deixa de ajudar) num problema real de processamento de imagens.

O **filtro Gaussiano** é uma técnica clássica de suavização que substitui cada pixel pelo resultado de uma média ponderada dos seus vizinhos, usando pesos que seguem uma distribuição Gaussiana. Aplicado repetidamente, o filtro produz um efeito de desfoque progressivo. É amplamente usado em visão computacional como pré-processamento antes de detectores de borda, por exemplo.

O **OpenMP** (*Open Multi-Processing*) é uma API para paralelismo em memória compartilhada em C, C++ e Fortran. Com diretivas de compilador (`#pragma omp ...`), é possível paralelizar laços de forma relativamente simples, sem precisar gerenciar *threads* manualmente.

Analisar o desempenho é importante porque paralelizar um código nem sempre traz o ganho esperado. Fatores como acesso à memória, arquitetura do processador e tamanho do problema influenciam muito o resultado. Neste projeto medimos *speedup* e eficiência para entender esses efeitos de forma quantitativa.

2 Metodologia

2.1 Implementação sequencial

A versão serial carrega a imagem em escala de cinza como um vetor 1D de `float`, indexado por `img[i * width + j]`. O filtro é aplicado com dois buffers (`src` e `dst`) que são alternados a cada iteração via troca de ponteiros, evitando alocações extras. Os pixels de borda são copiados sem filtragem; o filtro atua apenas nos pixels internos.

O kernel 3×3 é gerado a partir do parâmetro σ e normalizado para que a soma dos pesos seja 1, o que elimina a divisão dentro do laço de convolução:

$$G(x, y) = \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (1)$$

Com $\sigma = 1,0$ isso gera o kernel clássico $\{1, 2, 1\} / \{2, 4, 2\} / \{1, 2, 1\} \div 16$.

```
1 for (int i = 1; i < height - 1; i++) {
2     for (int j = 1; j < width - 1; j++) {
3         float sum = 0.0f;
4         for (int k = -1; k <= 1; k++)
5             for (int l = -1; l <= 1; l++)
6                 sum += src[(i+k)*width + (j+l)] * kernel[k+1][l+1];
7         dst[i*width + j] = sum;
8     }
9 }
```

Listing 1: Loop serial de convolucao

2.2 Implementação paralela com OpenMP

Implementamos duas versões paralelas. A primeira (**OpenMP simples**) é a implementação exigida pelo projeto. A segunda (**tiling**) foi desenvolvida por curiosidade, como uma otimização extra não obrigatória, para investigar se organizar o trabalho em blocos 2D traria ganho adicional de desempenho.

2.2.1 OpenMP simples

Adicionamos `#pragma omp parallel for` no laço externo da convolucao. O OpenMP divide as linhas da imagem entre as *threads* automaticamente com escalonamento estático (blocos contíguos iguais). É a abordagem mais direta de paralelizar o problema e não exige nenhuma mudança na lógica do código.

```
1 #pragma omp parallel for
2 for (int i = 1; i < height - 1; i++) {
3     for (int j = 1; j < width - 1; j++) {
4         /* mesma convolucao */
5     }
6 }
```

Listing 2: Paralelizacao com OpenMP simples

2.2.2 Tiling (blocos 2D)

Na versão com tiling, dividimos a imagem em blocos 64×64 pixels e paralelizamos os próprios blocos:

```
1 #define TILE 64
2 #pragma omp parallel for collapse(2) schedule(dynamic)
3 for (int ti = 1; ti < height - 1; ti += TILE) {
4     for (int tj = 1; tj < width - 1; tj += TILE) {
5         /* processa o bloco [ti..ti+TILE][tj..tj+TILE] */
6     }
7 }
```

Listing 3: Paralelizacao com tiling 2D

Usamos `collapse(2)` para que o OpenMP trate os dois laços como um só espaço de iterações, aumentando o número de tarefas distribuíveis. O `schedule(dynamic)` ajuda a equilibrar a carga dos blocos de borda, que são menores que os demais. A motivação do tiling é melhorar a localidade de cache: cada *thread* trabalha em uma região compacta da memória, reduzindo falhas de cache.

2.3 Hardware utilizado

Os testes foram executados em um processador **AMD Ryzen 7 5700G**, que possui 8 núcleos físicos com SMT (*Simultaneous Multi-Threading*), totalizando 16 *threads* lógicas visíveis ao sistema operacional. É um processador de nó único, com 16 MB de cache L3 compartilhado entre todos os núcleos. O ambiente é Linux (WSL2), compilador GCC com flags `-O2 -Wall -fopenmp`.

2.4 Medição de tempo e métricas

O tempo de execução é medido com `omp_get_wtime()`, que retorna o tempo de parede (*wall-clock time*) com resolução de nanosegundos. Cada configuração roda 5 vezes e reportamos a média.

As métricas usadas são:

- **Speedup:** $S = T_{\text{serial}}/T_{\text{paralelo}}$
- **Eficiência:** $E = S/N_{\text{threads}} \times 100\%$

Os testes cobriram 4 tamanhos de imagem (512^2 , 1024^2 , 2048^2 , 4096^2 pixels), com 2, 4, 8 e 16 *threads*, 100 iterações do filtro e $\sigma = 1,0$.

3 Resultados

3.1 Teste de corretude

Para verificar que as versões paralelas produzem o mesmo resultado que a serial, comparamos visualmente as imagens de saída geradas por cada implementação. Os arquivos `output_serial.png`, `output_openmp_Nt.png` e `output_tiling_Nt.png` apresentaram resultado idêntico para todas as configurações testadas, confirmando a corretude das implementações paralelas. Isso era esperado, pois as três versões usam o mesmo kernel e a mesma ordem de cálculo dos pixels internos (sem dependência entre linhas).

3.2 Tempos seriais

Tabela 1: Tempos de execução serial (média de 5 repetições)

Imagem	Tempo médio (s)
512×512	0,0814
1024×1024	0,3301
2048×2048	1,3305
4096×4096	5,5616

3.3 OpenMP — Tempos, Speedup e Eficiência

Tabela 2: Resultados OpenMP

Imagem	Threads	Tempo (s)	Speedup	Eficiência (%)
512×512	2	0,0590	$1,38\times$	69,1
	4	0,0311	$2,62\times$	65,5
	8	0,0396	$2,06\times$	25,7
	16	0,0304	$2,68\times$	16,8
1024×1024	2	0,2595	$1,27\times$	63,6
	4	0,1392	$2,37\times$	59,3
	8	0,1474	$2,24\times$	28,0
	16	0,1025	$3,22\times$	20,1
2048×2048	2	0,9648	$1,38\times$	69,0
	4	0,4889	$2,72\times$	68,0
	8	0,5907	$2,25\times$	28,2
	16	0,4680	$2,84\times$	17,8
4096×4096	2	4,3316	$1,28\times$	64,2
	4	2,3367	$2,38\times$	59,5
	8	2,3802	$2,34\times$	29,2
	16	1,6097	$3,45\times$	21,6

3.4 Tiling — Tempos, Speedup e Eficiência

Tabela 3: Resultados Tiling

Imagem	Threads	Tempo (s)	Speedup	Eficiência (%)
512×512	2	0,0645	$1,26\times$	63,1
	4	0,0336	$2,43\times$	60,7
	8	0,0307	$2,65\times$	33,2
	16	0,0343	$2,37\times$	14,8
1024×1024	2	0,2845	$1,16\times$	58,0
	4	0,1339	$2,47\times$	61,7
	8	0,1000	$3,30\times$	41,3
	16	0,0957	$3,45\times$	21,6
2048×2048	2	1,0463	$1,27\times$	63,6
	4	0,5556	$2,39\times$	59,8
	8	0,4287	$3,10\times$	38,8
	16	0,3775	$3,52\times$	22,0
4096×4096	2	4,5239	$1,23\times$	61,5
	4	2,5597	$2,17\times$	54,3
	8	1,7743	$3,14\times$	39,2
	16	1,4727	$3,78\times$	23,6

3.5 Gráficos

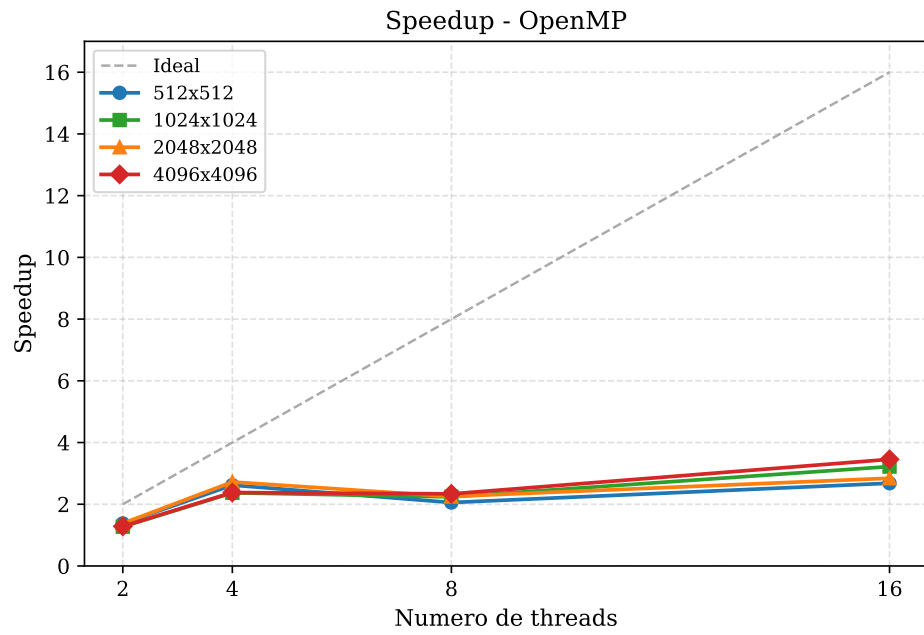


Figura 1: Speedup do OpenMP em função do número de *threads*

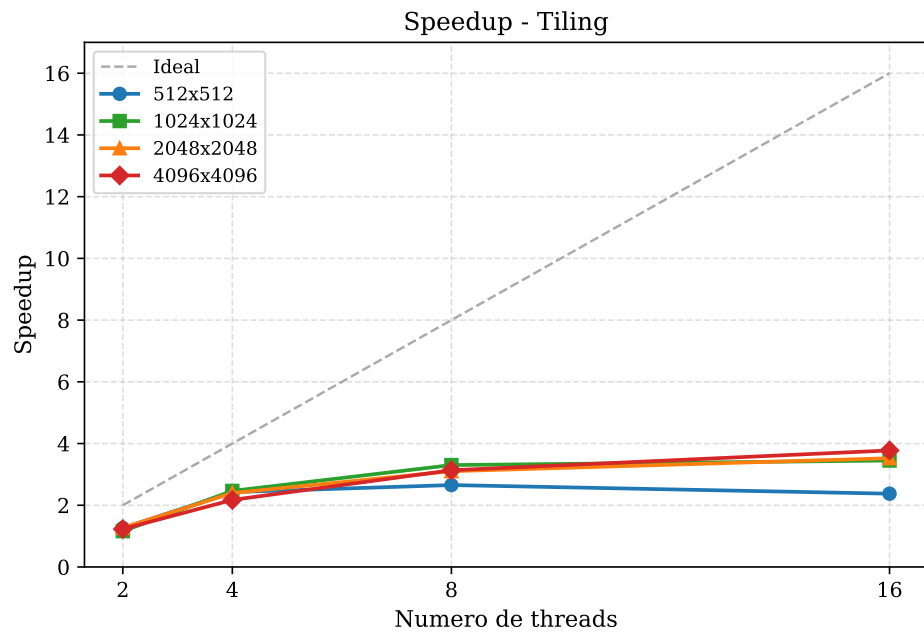


Figura 2: Speedup do Tiling em função do número de *threads*

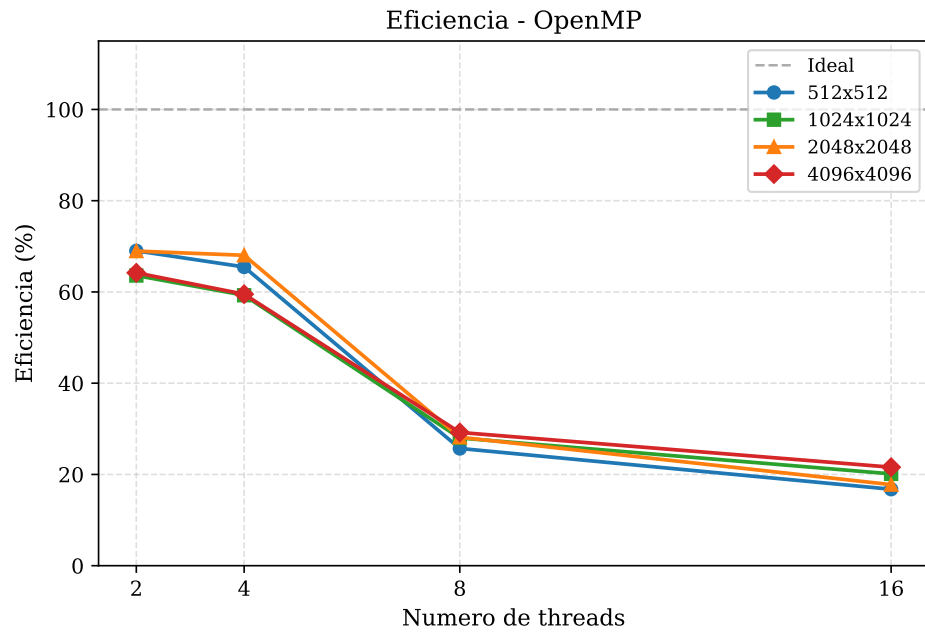


Figura 3: Eficiência do OpenMP em função do número de *threads*

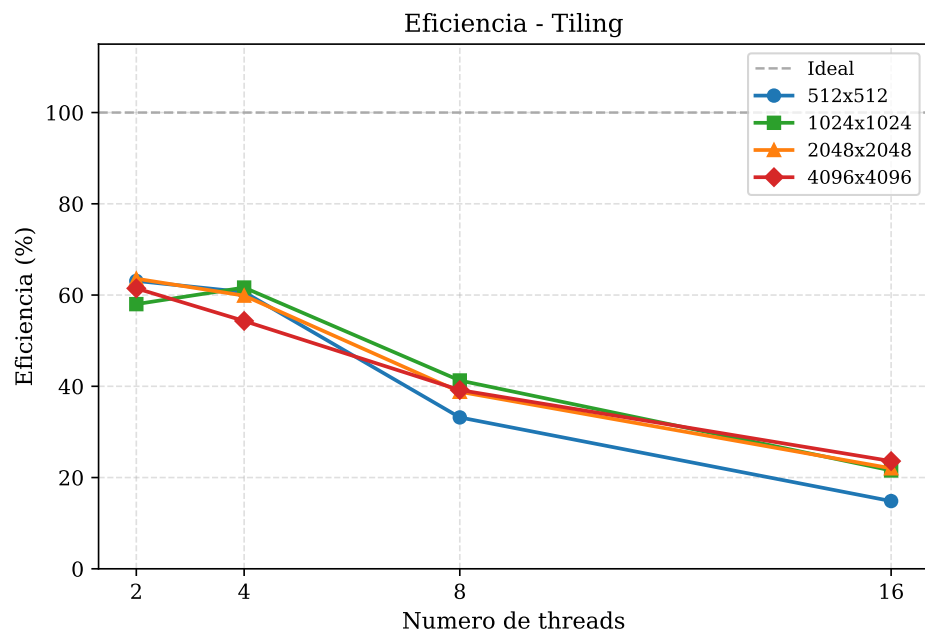


Figura 4: Eficiência do Tiling em função do número de *threads*

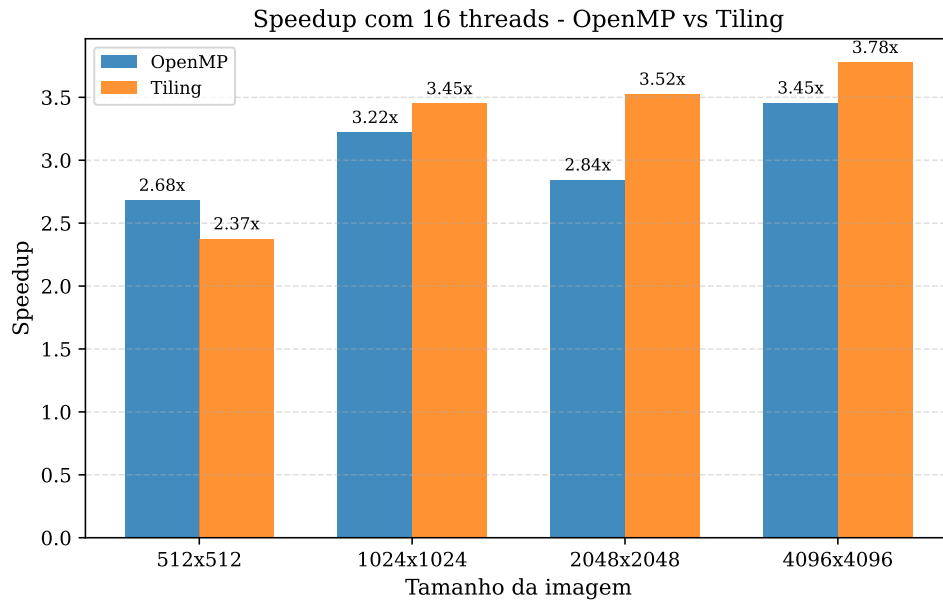


Figura 5: Comparação de speedup a 16 *threads*: OpenMP vs Tiling

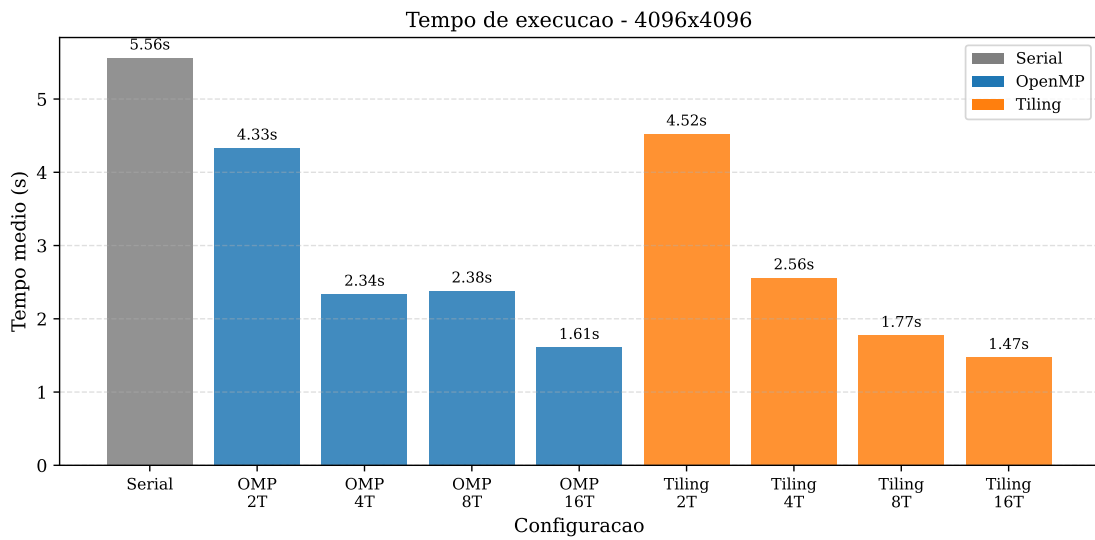


Figura 6: Tempo de execução para a imagem 4096×4096

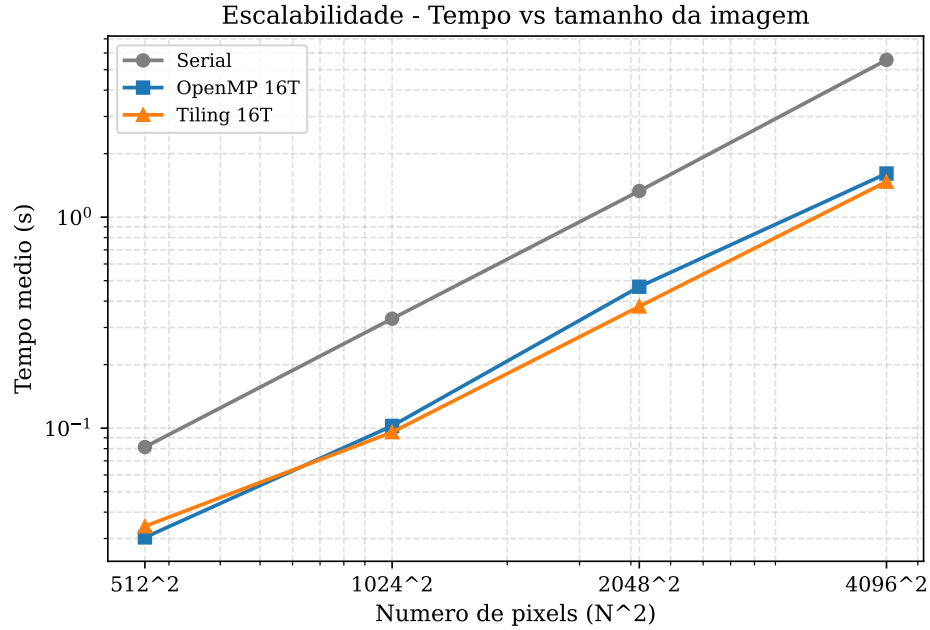


Figura 7: Tempo de execução em função do tamanho da imagem (escala log-log)

3.6 Balanceamento de carga e escalabilidade

A fração paralela do trabalho é muito alta: os pixels internos representam 99,2% do total para 512^2 e chegam a 99,9% para 4096^2 . A parte serial (cópia das bordas) é praticamente desprezível.

Em relação à escalabilidade, o speedup melhora conforme o tamanho da imagem aumenta, o que faz sentido: com imagens maiores, cada *thread* tem mais trabalho e o custo fixo de criar e sincronizar as *threads* representa uma fração menor do tempo total.

4 Discussão

4.1 Corretude das implementações

Como mencionado nos resultados, todas as versões produzem imagens visualmente idênticas. Isso é esperado porque cada pixel interno é calculado de forma independente a partir dos mesmos vizinhos em *src* — não há dependência entre os pixels de uma mesma iteração, então a ordem de cálculo não altera o resultado.

4.2 Comparação serial vs paralela

De forma geral, o OpenMP trouxe ganhos reais, mas bem abaixo do ideal teórico. Com 16 *threads* na imagem 4096×4096 , o melhor resultado foi $3,78\times$ de speedup (tiling), quando teórico seria próximo de $16\times$.

Para imagens pequenas como 512×512 , o ganho é ainda menor. O problema é que o trabalho por *thread* é pequeno demais e o *overhead* do OpenMP (criar *threads*, sincronizar, etc.) come boa parte do tempo. Já com 4096×4096 , o trabalho é suficientemente grande para que o paralelismo compense melhor.

4.3 Impacto do número de *threads*

Um resultado que chama a atenção é que com **8 *threads* o desempenho piorou em relação a 4** em várias configurações OpenMP. O Ryzen 7 5700G tem exatamente 8 núcleos físicos, então com 8 *threads* todos os núcleos estão ativos simultaneamente e competindo pela mesma banda de memória e pelo cache L3. O filtro Gaussiano é um workload *memory-bound*: cada pixel exige ler 9 vizinhos da memória e escrever 1 resultado, e para imagens grandes o conjunto de dados não cabe no cache. A partir de certo ponto, adicionar mais *threads* não acelera o cálculo — o gargalo vira a banda de memória, e o *overhead* de sincronização passa a pesar mais. Com 16 *threads* (SMT), dois *threads* por núcleo conseguem intercalar requisições de memória, escondendo parte da latência e recuperando algum desempenho. Esse efeito é visível nos gráficos de speedup (Figuras 1 e 2).

O tiling se sai melhor nessa situação porque cada *thread* trabalha num bloco 2D compacto, melhorando o reuso do cache e reduzindo a pressão sobre a memória principal. Por isso o tiling supera o OpenMP simples especialmente com 8+ *threads* em imagens grandes.

4.4 Gargalos e limitações

O principal gargalo não é o cálculo em si, mas o acesso à memória. Cada pixel precisa ler 9 vizinhos de `src` e escrever em `dst`, e para imagens de 4096×4096 os dois buffers ocupam juntos cerca de 128 MB — muito além dos 16 MB de cache L3 do processador. Com múltiplas *threads* acessando regiões distintas da memória ao mesmo tempo, a banda de memória satura e vira o limitante real do desempenho.

Outra limitação é a alta variância observada em algumas execuções (notável, por exemplo, na primeira repetição de vários testes com 16 *threads*), provavelmente causada por interferência do sistema operacional no agendamento das *threads*.

4.5 Sugestões de melhorias

- Experimentar outros tamanhos de bloco no tiling (32, 128) para encontrar o melhor ajuste para o cache L2/L3 do processador.
- Fixar a afinidade das *threads* com `OMP_PROC_BIND=close` para garantir que *threads* vizinhas fiquem em núcleos próximos e compartilhem melhor o cache.
- Usar instruções SIMD (AVX2) para vetorizar o laço interno da convolução, o que poderia aumentar bastante o *throughput* de cálculo e reduzir a pressão sobre a memória.

5 Conclusão

Neste projeto implementamos e comparamos três versões de um filtro Gaussiano iterativo: serial, OpenMP e OpenMP com tiling. Os resultados mostraram que o paralelismo funciona, mas a banda de memória do processador limita bastante o ganho obtido.

O aprendizado mais importante foi perceber que paralelizar um laço nem sempre é suficiente: a forma como os dados são acessados na memória é tão importante quanto

a divisão do trabalho entre as *threads*. O tiling mostrou isso claramente ao superar o OpenMP simples em imagens grandes com muitas *threads*.

Outro ponto relevante foi a queda de desempenho com 8 *threads*, que só fez sentido depois de entender que o filtro é *memory-bound* e que a banda de memória satura quando todos os núcleos físicos estão ativos. Esse tipo de detalhe não fica óbvio olhando só para o código; é necessário medir e analisar.

No geral, o desempenho obtido ficou dentro do esperado para esse tipo de workload memory-bound: speedups entre $2\times$ e $4\times$ com 16 *threads*, com eficiência caindo com o aumento do número de *threads*, como previsto pela Lei de Amdahl.

6 Referências

- Material de aula da disciplina DEC107 — Processamento Paralelo.
- OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 5.2*. Disponível em: <https://www.openmp.org/specifications/>
- NOTHINGS. *stb_image / stb_image_write* — single-file public domain libraries for C/C++. Disponível em: <https://github.com/nothings/stb>